

SPRINTHUB: TECHNICAL IMPLEMENTATION SPEC

Architecture and Logic for VS Code Extension v2.0

PHASE 1: THE LOCAL-REMOTE BRIDGE

Contextual 'Work-On-This' Engine

TECHNICAL ARCHITECTURE:

Implement a matching algorithm that parses Issue IDs from card titles. Use the VS Code Workspace API to verify if a local branch containing that ID exists. If not, utilize the GitHub REST API to fetch the branch name associated with the PR.

IMPLEMENTATION SEQUENCE:

- Listener for dashboard click events.
- Executes: ``git rev-parse --verify [branch_name]`` via `child_process`.
- Triggers internal VS Code command: ``workbench.action.terminal.sendSequence`` to automate checkout/pull.
- Focuses editor on the most recently modified file (mtime check).

Intelligent File-Level Deep Linking

TECHNICAL ARCHITECTURE:

Parse PR file diffs via ``GET /repos/{owner}/{repo}/pulls/{pull_number}/files``. Store the ``filename`` and ``patch`` data.

IMPLEMENTATION SEQUENCE:

- Map filenames to local absolute paths using ``vscode.workspace.findFiles``.
- Use ``vscode.window.showTextDocument`` with a ``TextDocumentShowOptions`` selection range to jump to the first changed line in the PR.

PHASE 2: PREDICTIVE CONFLICT ANALYSIS

Active Workspace 'Hotspot' Monitor

TECHNICAL ARCHITECTURE:

Poll open PRs every 5-10 minutes to extract modified file lists. Compare these against the user's local ``git diff`` status.

IMPLEMENTATION SEQUENCE:

- Intersection check between Local Dirty Files and Remote PR file sets.
- Trigger 'Conflict Threat' decoration in the SprintfHub dashboard sidebar if an overlap is detected.
- Display specific PR # and Author responsible for the 'competing' file changes.

Status Bar 'Blocker' Watchdog

TECHNICAL ARCHITECTURE:

Utilize the ``vscode.StatusBarItem`` API to create a persistent indicator. Connect to GitHub Webhooks or poll the Checks API (``GET /repos/{owner}/{repo}/commits/{ref}/check-runs``).

IMPLEMENTATION SEQUENCE:

- Aggregate CI status and 'Changes Requested' counts.
- Implement a hover provider to show a quick-pick list of failing builds directly from the status bar.

PHASE 3: AUTOMATED REPORTING & METADATA

Standup Markdown Generator

TECHNICAL ARCHITECTURE:

Query the user's GitHub activity for the last 24h: ``GET /search/issues?q=repo:{repo}+is:pr+is:merged+merged:>{today-1}``.

IMPLEMENTATION SEQUENCE:

- Format result as a Markdown list template.
- Integration with ``vscode.env.clipboard`` to allow instant pasting into Slack/Teams.

Metadata Sync (Labels/Assignees)

TECHNICAL ARCHITECTURE:

Direct integration with ``PATCH /repos/{owner}/{repo}/issues/{issue_number}``.

IMPLEMENTATION SEQUENCE:

- Implement custom UI components (dropdowns/badges) within the WebView.
- Optimistic UI updates to show the label change instantly while the API call completes in the background.